

Subject:

Date:

Chap - 22 (NLP)

NLP is a branch of AI that helps to communicate with intelligent system using human language instead of programming language.



Purpose of NLP → →

i) Language Understanding: Help computers understand human language. Ex: Google search, Siri

ii) Language Generation: make computers generate text
Ex: Chat GPT writes essay, paragraph

iii) Machine Translation: Translate one language to another. Ex: Google translator

iv) Sentiment Analysis: detect emotion or opinion
Ex: social media review

Subject:.....

Date:.....

v) speech recognition: Convert spoken language to text. Ex: Voice assistant (Siri)

vi) Text summarization: large text → short text (meaningful)

vii) Question Answering: answer question asked in natural language.

viii) Chatbots: conversational system respond like

human.

Ex: ChatGPT

N-gram character model

N-gram character model is a sequence of N characters that appear together.

⇒ 1-gram (Unigram) → one character ("a")

⇒ 2-gram (Bigram) → two characters ("th")

⇒ 3-gram (Trigram) → three characters ("the")

A N-gram model learns how often these character groups appear in a language, helping to predict the next character.

* A N-gram model is defined as a Markov chain of order $N-1$.

↳ next character depends on the previous $N-1$ characters.

Bigram → previous one Trigram → previous 2

$$P(c_1:N) = \prod_{i=1}^N P(c_i | c_{1:i-1}) = \prod_{i=1}^N P(c_i | c_{i-2:i-1})$$

(Trigram)

Uses

- i) Language identification ($\rightarrow$ detect language)
- ii) Spelling Correction
- iii) Genre Classification ($\rightarrow$ identify the type of text (news, legal, scientific))
- iv) Named Entity Recognition (NER) ($\rightarrow$ find Names of people, drugs, place etc.)

Smoothing n-gram models

Problem: When we make an N-gram model, we count how often words or letters come together.

$\Rightarrow$ But sometimes, in our training text, some word combinations never appear.

Ex: "th" $\rightarrow$ this, that (common)

But "ht" $\rightarrow$ maybe never show

model makes its, probability = 0

But "http" word is available in reality.

$$p(c_1, c_2, \dots, c_N) = \prod_{i=1}^N p(c_i | c_{i-1}, \dots, c_1)$$

Solution: $\rightarrow$ Smoothing

$\hookrightarrow$ we don't let anything have probability 0

Types of Smoothing

1) Laplace Smoothing: Just add 1 to every count

Ex: 'ht' appear 0 times, make it 1 time.

$\rightarrow$ Simple but not very accurate.

2) Backoff model: If big model doesn't know something then backs off to smaller model.

Try 3-gram $\rightarrow$ "The cat" if missing

" 2-gram $\rightarrow$ "cat"

" 1-gram $\rightarrow$ "the"

3) Linear Interpolation: Combining trigram, bigram, unigram together.

$$P(c_i | c_{i-2:i-1}) = \lambda_3 P(c_i | c_{i-2:i-1}) + \lambda_2 P(c_i | c_{i-1}) + \lambda_1 P(c_i)$$

where, $\lambda_3 + \lambda_2 + \lambda_1 = 1$

[$\lambda \rightarrow$ importance scores]

Model evaluation

There are many model ways to build n-gram model (unigram, bigram, trigram). We need to know which one works best.

↳ solution model evaluation.

Step-1: Split the corpus

⇒ Training set — Teach the model

⇒ Validation set — Test how well it learned

Step-2: how to measure "goodness"

→ option-1: Task-based evaluation —

Ex: language identification

→ Accuracy predicts (%)

→ option-2: Probability-based evaluation —

→ give probability for validation test

→ Higher probability → better model

Problem → for long texts probability became tiny

→ solution: perplexity

Step-3 : Perplexity

$$\text{perplexity}(c_1:N) = \frac{1}{P(c_1:N)^{1/N}}$$

Model predicts well $\rightarrow$ probability $\uparrow$ perplexity $\downarrow$

Model confused $\rightarrow$ probability $\downarrow$ perplexity $\uparrow$

N-gram Word model

"Which word is likely come after another word?"

$\Rightarrow$ Unigram $\rightarrow$ single words ("The", "cat")

$\Rightarrow$ Bigram $\rightarrow$ 2 " ("The cat", "runs fast")

$\Rightarrow$ Trigram $\rightarrow$ 3 " ("The cat runs", "cat runs fast")

problem: Word model can see new words, that not in the training model.

Solution: Use unknown token $\langle \text{UKN} \rangle$ which is replace with unknown word.

Then calculate probability.

Ex:

<u>Sentence</u>	<u>Model</u>
I love pizza	I love pizza
I love xyzburgerz	I love <UNK>

→ give perfect result in Trigram.

Perplexity : Trigram > bigram > unigram

Text classification

Text classification is the process of teaching a computer to "automatically" sort a piece of text into a category like spam, positive/negative or news.

Example : Spam Detection

⇒ split emails into two folders

→ spam folder → all spam example

→ Inbox (Ham) → all real emails

Subject:

Date:

⇒ clues for spam

<u>Spam</u>	<u>Ham</u>
"Buy now", "Cheap", "Viagra"	"Meeting at 3 pm"
All uppercase (ALL CAPS)	Normally written
weirdly written (you-d-serve)	Normal words

⇒ Computer build two model

$$P(\text{Message} | \text{spam}), P(\text{Message} | \text{ham})$$

use bayes rule,

$$\underset{c \in \{\text{spam}, \text{ham}\}}{\text{argmax}} P(c | \text{message}) = \underset{c \in \{\text{spam}, \text{ham}\}}{\text{argmax}} P(\text{message} | c) P(c)$$

⇒ counting total ham and spam message

⇒ result

Information Retrieval (IR)

Finding documents that are relevant to a user's information need.

Ex: In google — [AI Book]

Components

i) Corpus of documents: The collection of documents the system search.

Ex: a paragraph, a page, text, file

ii) Query in a query language: Specifies what the user wants.

Ex: [AI book] — keyword

iii) Result set: System thinks which documents is suitable for search.

iv) Presentation: How results shown. list or visualization.

IR Scoring Functions

→ a methods used by search engines to rank documents based on how relevant they are to a user's query.

→ BM25 is one of the best.

BM25: a formula that gives each search page a score.

High score → more relevant

BM25 three factors →

i) TF → How many times a word appears

Ex: If a query is "farming"

Page A: 10 times ✓

Page B: 1 time X

ii) IDF → How rare a word is!

Ex:

'in' — appears everywhere — low importance

'farmers' — rare — high importance

iii) length documents →

long page may mention all words by chance → low score

Short, focused pages mention all query word → high score.

$$\text{BM25}(d_j, q_1:N) = \sum_{i=1}^N \text{IDF}(q_i) \cdot \frac{\text{TF}(q_i, d_j) \cdot (k+1)}{\text{TF}(q_i, d_j) + k \cdot (1 - b + b \cdot \frac{|d_j|}{L})}$$

where,

$|d_j|$ = length of document d_j

L = avg document length in the corpus

$$L = \sum_i \frac{|d_i|}{N}$$

$\left. \begin{array}{l} k = 2 \\ b = 0.75 \end{array} \right\}$ parameters

$$\text{IDF}(q_i) = \log \frac{N - \text{DF}(q_i) + 0.5}{\text{DF}(q_i) + 0.5}$$

Subject:

Date:

IR System evolution

The Page Rank algorithm

PageRank is a Google algorithm that measures

which web page is most important.

The HITS algorithm —

function HITS (query) returns pages with high

authority numbers

Pages $\leftarrow$ EXPAND-PAGES (RELEVANT-PAGES) (query)

for each p in Pages do

$p.AUTHORITY \leftarrow 1$

$p.HUB \leftarrow 1$

repeat until convergence do

for each p in Pages do

$p.AUTHORITY \leftarrow \sum_i INLINK_i(p).HUB$

$p.HUB \leftarrow \sum_i OUTLINK_i(p).AUTHORITY$

NORMALISE (pages)

return pages

The Page rank algorithm

PageRank is a Google's algorithm that decides which web page is most important. ~~for~~

The HITS algorithm —

function HITS(query) returns pages with hub and authority numbers

$Pages \leftarrow EXPAND_PAGES(RECENT_PAGES)(query)$

for each p in $Pages$ do

$p.AUTHORITY \leftarrow 1$

$p.HUB \leftarrow 1$

repeat until convergence do

for each p in $Pages$ do

$p.AUTHORITY \leftarrow \sum_i INLINK_i(p).HUB$

$p.HUB \leftarrow \sum_i OUTLINK_i(p).AUTHORITY$

NORMALIZE($Pages$)

return $Pages$

The PageRank for a page p is defined as:

$$PR(p) = \frac{1-d}{N} + d \sum \frac{PR(i)}{C(i)}$$

where

N = Total Pages

d = damping factor (≈ 0.85)

i = Page linking to page p

$C(i)$ = number of outgoing links on page i